5COSC019W · OBJECT ORIENTED PROGRAMMING

Lecture 2

Encapsulation & UML Class Diagrams

BSc Computer Science · Level 5 · 2 Hour Lecture

LEARNING OUTCOMES

LO1 Identify and justify core OOP design principles in Java

LO5 Communicate design using professional UML notation

Builds directly on Lecture 1's Product class



Dr Barbara Villarini

SESSION ROADMAP

Today's Agenda

01

Access Modifiers in Depth

The four levels — and why fields should almost always be private

02

Why Private Fields Matter

How an object becomes JSON — then we break it on purpose

03

Getters, Setters & JavaBeans

Naming convention, and designing for immutability

04

Constructors — Full vs No-Arg

Why Jackson needs a constructor it can call with nothing

05

UML Class Diagram Notation

The professional language for expressing class design

06

From Diagram to Code and Back

Design starts on paper, not in the IDE

01

Access Modifiers in Depth

20 minutes · private, default, protected, public

LO1

The Four Access Levels



`private`

Only inside this class



`default`

Only inside this package



`protected`

Package + subclasses

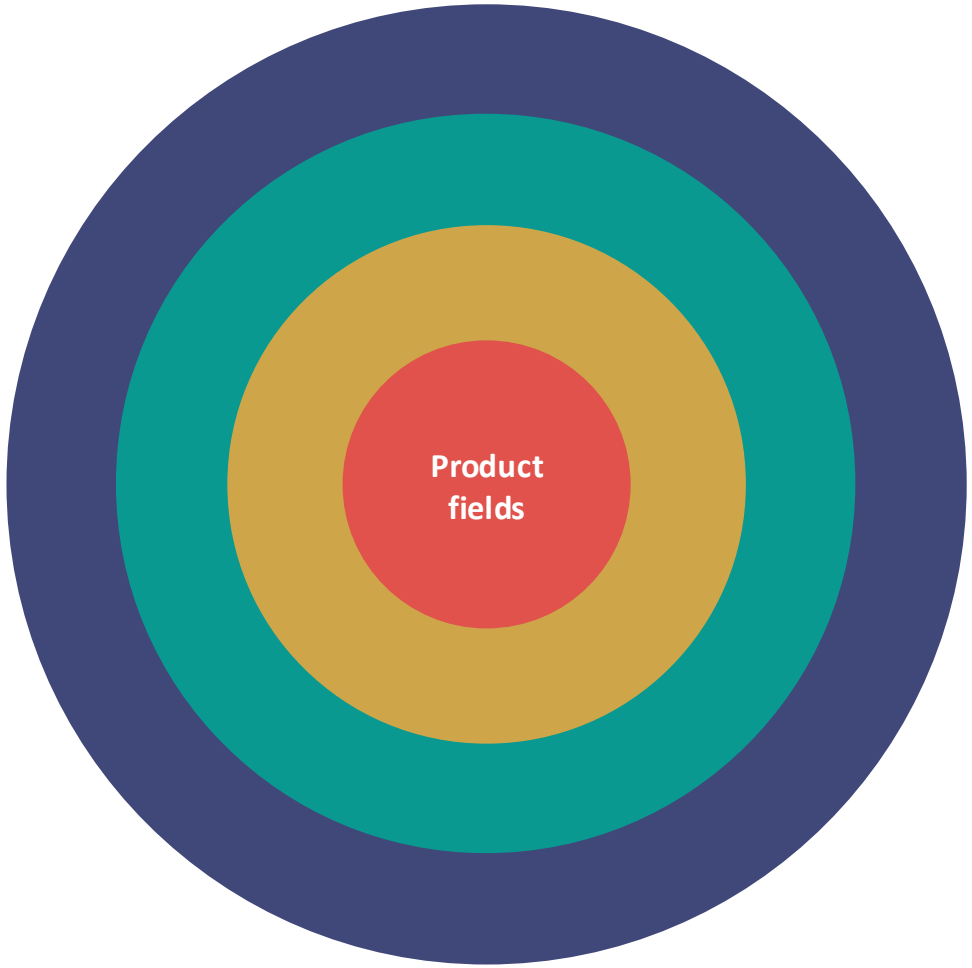


`public`

Anywhere, no restriction

RULE OF THUMB Fields should almost always be private. Methods that are part of the API should be public.

Visualising Scope: Who Can See What?



`private`

Same class only — the safest, most common choice for fields

`default`

Same package — no keyword needed, easy to forget it's restrictive

`protected`

Same package, plus any subclass elsewhere

`public`

Every class, everywhere — reserve for your real API

LIVE CODE

LIVE

Try It: What Happens If...?

```
public class Product {
    private Long id;
    private String name;
    private double price;

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
}

// In another class, same package:
Product p = new Product(1L, "Laptop", 999.99);
System.out.println(p.name);
// ❌ COMPILE ERROR — 'name' has private access
```

ASK THE ROOM

"Why does the compiler stop us here — and is that a good thing?"

- Java is protecting the object's internal state
- Only Product's own code should touch these fields directly
- This is encapsulation enforced by the compiler, not just convention

02

Why Private Fields Matter
The Spring Connection

30 minutes · from request to JSON, then breaking it

LO1 LO2

Two Things You Built — That Have Never Met

```java
public class Product {
    Long   id;
    String name;
    double price;

    public Product(Long id,
                   String name,
                   double price) {
        this.id    = id;
        this.name  = name;
        this.price = price;
    }
}
```

Lecture 1, Section 03 · Classes & Objects

```java
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Spring Boot!";
    }
}
```

Lecture 1, Section 05 · Your First REST Endpoint

A class that models data, and a controller that answers HTTP. Today the controller starts returning a Product instead of a String.

What a Request Asks For: the Verbs

GET

`@GetMapping`

Read something. Changes nothing on the server.

```
GET /products/1
```

POST

`@PostMapping`

Create something new from data you send.

```
POST /products
```

PUT

`@PutMapping`

Replace an existing item with what you send.

```
PUT /products/1
```

DELETE

`@DeleteMapping`

Remove an item from the collection.

```
DELETE /products/1
```

Every request is a verb plus a path. The verb says what to do; the path says what to do it to.

Today we use GET only. POST arrives in Section 04 with @RequestBody, and the full set in Lecture 3.

Where GET /products/1 Comes From

```java
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public Product getById(
            @PathVariable Long id) {
        return new Product(
            id, "Laptop", 999.99);
    }
}
```

ProductController.java · new this week

GET

The verb from the previous slide — read this resource, change nothing. Typing the URL in a browser sends exactly this.

/products

The path. @RequestMapping("/products") on the class claims every URL that starts this way — new alongside @GetMapping.

/1

The path variable. The braces in @GetMapping("/{id}") mark a slot, and @PathVariable copies the 1 into the id parameter.

Try it yourself: http://localhost:8080/products/1 in the browser, or press Try it out on this endpoint in Swagger UI.

What Is JSON — and Why Send Text at All?

```
{
  "id": 1,
  "name": "Laptop",
  "price": 999.99
}
```

Reading it:

{ } wraps one object

"key" : value is one field of that object

text is quoted, numbers are bare, fields are comma-separated

JavaScript Object Notation

A plain-text way of writing down structured data. Despite the name it does not run anything and is not tied to JavaScript — it is simply an agreed layout for values.

Why it has to become text

Remember the heap diagram from Lecture 1: p1 held a reference to an object in memory. A network cable cannot carry memory, and an HTTP response body carries text — so the object has to be written out.

Why JSON rather than Java

The caller might be a React page, a Python script, an Android app or Swagger UI. JSON is language-neutral, so every one of them can read the same response.

The field names come from your Java class — but the format itself belongs to the whole web, not to Java.

THE MISSING STEP

Serialisation: Object → Text

1

Your method returns an object

```
new Product(1L, "Laptop", 999.99)
```

a real object, held in memory

2

Spring writes it out as text

```
"name" : "Laptop"
```

done for you by a library that came with starter-web

3

The client receives characters

```
{ "id": 1, "name": ... }
```

carried in the HTTP response body

The same machinery runs in reverse — deserialisation: JSON arriving in a POST body is turned back into a Product object. That is Section 04.

Everything that follows turns on one question: when Spring writes your object out, how does it read the values?

Where We Left Off: Public Fields Work Fine...

```
public class Product {
    public Long id;
    public String name;
    public double price;
    ...
}
```

GET /products/1

```
{
  "id": 1,
  "name": "Laptop",
  "price": 999.99
}
```

✅ *The JSON looks perfect. So what's the problem?*

The problem isn't visible yet — it's a design smell waiting to bite. Let's find out why.

LIVE DEMO

LIVE

Breaking It — Live, On the Projector

```
public class Product {
    private Long id;
    private String name;
    private double price;

    public Product() {}
    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
    // no getters yet...
}
```

GET /products/1

```
{ }
```

Empty JSON. No error thrown — just... nothing.

WHY THIS MATTERS

This is the clearest possible demonstration that encapsulation isn't just a style rule — it has real, visible consequences in a running Spring app.

The Diagnosis: Jackson Needs Getters



Jackson

Spring uses a library called **Jackson** to convert your Java objects into JSON.
Jackson does not read private fields directly — it looks for a public `getX()` method for each field it wants to serialise.

No getter → Jackson finds nothing to call → that field is silently left out of the JSON.

You already have Jackson — it arrived inside spring-boot-starter-web, the Spring Web box you ticked on start.spring.io in Lecture 1.

The Fix: Restore the Getters

❌ **BEFORE**

```
private String name;
// no getter
```

Result: "name" missing from every response

✅ **AFTER**

```
private String name;
public String getName() {
    return name;
}
```

Result: "name" appears correctly in the JSON

KEY TAKEAWAY

Encapsulation is not just a rule. It has real consequences.

`private` fields protect your object's state — but Spring's JSON serialiser can only expose what a `public getter` reveals. Design your class API deliberately: private data, public behaviour.

03

Getters, Setters & the JavaBean Convention

Naming rules — and designing for immutability

LO1 LO2

The Naming Convention

`private String name;`	**`getName()`**	String getName() { return name; }
`private double price;`	**`getPrice() / setPrice()`**	void setPrice(double price) { this.price = price; }
`private boolean inStock;`	**`isInStock()`**	boolean isInStock() { return inStock; } (boolean uses is-, not get-)

To Setter, or Not to Setter?



Mutable field → include a setter

Use a setter only when the field genuinely needs to change from outside the class after construction.

e.g. a shopping cart's quantity field



Should never change → no setter

If a value is fixed once the object is created, don't offer a way to mutate it. Fewer setters means fewer ways for state to become inconsistent.

e.g. a Product's price never changes after creation

LIVE CODE

LIVE

Building an Immutable Product

```
public class Product {
    private final Long id;
    private final String name;
    private final double price;

    public Product() {} // required by Jackson
    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    // no setPrice() — price is fixed for the lifetime of the object
```

QUICK CHECK

Before We Move On...

Q1 **Why does Product need a no-arg constructor if it already has a full one?**

→ *Jackson uses it to construct the object before setting fields during deserialisation.*

Q2 **If a field has a getter but no setter, can it still change value?**

→ *Yes — inside the class itself. Outside code just can't change it directly.*

Q3 **What HTTP response would you expect from a private field with no getter?**

→ *It's simply omitted from the JSON — no error, no warning.*

04

Constructors Full vs No-Arg

Why Spring quietly needs both

LO1 LO2

Why Spring Often Needs a No-Arg Constructor

1

JSON arrives

```
{ "name": "Laptop", "price": 999.99 }
```

as a @RequestBody

2

Jackson builds it

```
new Product()
```

then calls each setter / assigns each field

3

Without a no-arg constructor...

```
new Product() has no matching constructor
```

→ *deserialisation fails at runtime*

LIVE DEMO

LIVE

@RequestBody Fails Without It

```
// Product with ONLY the full constructor:
public Product(Long id, String name, double price) {
    this.id = id;
    this.name = name;
    this.price = price;
}

@PostMapping
public Product addProduct(@RequestBody Product p) {
    return productService.addProduct(p);
}

// POST /products  { "name": "Mouse", "price": 19.99 }
// ❌ HTTP 400 — Jackson cannot construct Product
```

THE FIX

```
public Product() {}
```

Add this no-arg constructor alongside the full one — one line restores the endpoint.

Restart. Retry the POST. Watch it succeed.

The Pattern: Two Constructors, Two Purposes

```
public class Product {
    private Long id;
    private String name;
    private double price;

    public Product() {}                                    // for frameworks (Jackson)
    public Product(Long id, String name, double price) {   // for your own code
        this.id = id; this.name = name; this.price = price;
    }
}
```

No-arg constructor

Exists purely so frameworks can instantiate the object before populating it field by field.

Full constructor

What you actually call in your own code — guarantees every field is set at creation time.

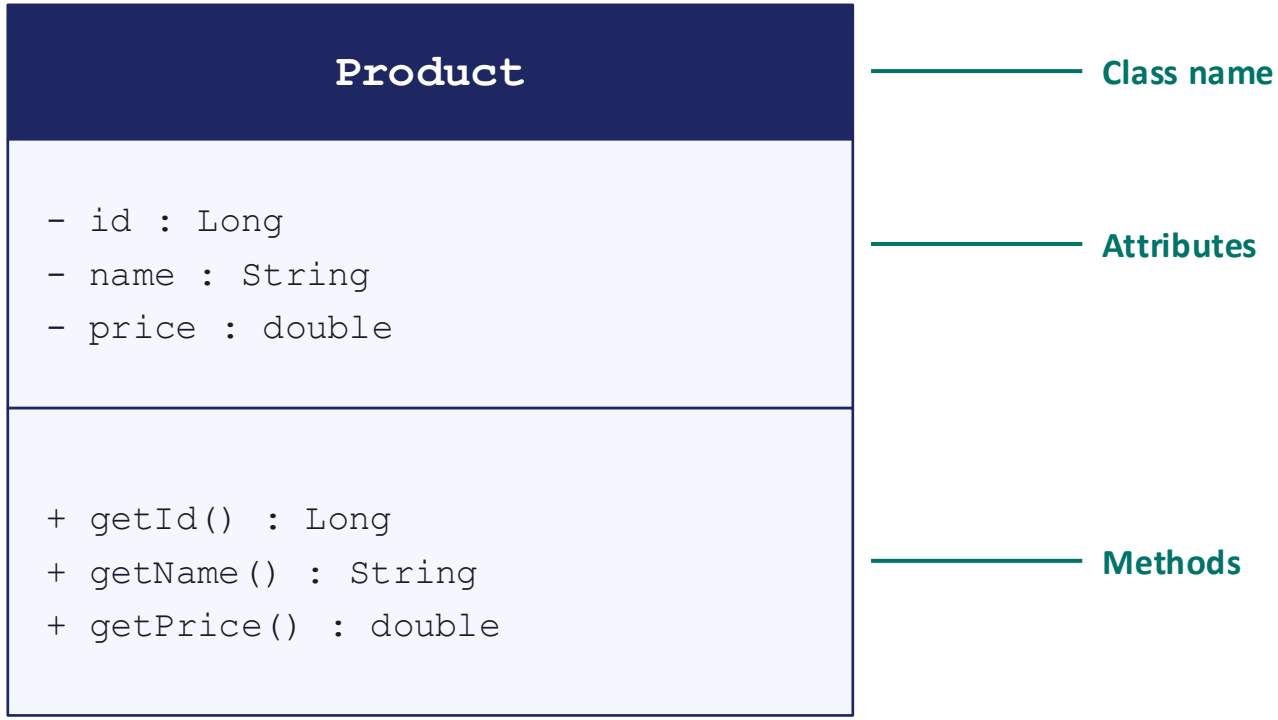
05

UML Class Diagram Notation

The professional language for class design

LO5

The Class Box: Three Compartments



REMEMBER

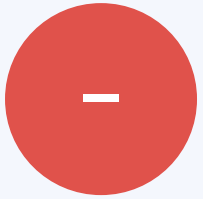
Every class box has the same 3 compartments, top to bottom, always.

This is the exact diagram used in your coursework report (LO5).

Visibility Symbols



`public`



`private`



`protected`



`default / package`

`- name : String` *reads as: "a private attribute called name, of type String"*

Attribute & Method Syntax

Attributes

```
visibility  name  :  Type
```

```
- id : Long
- name : String
- price : double
```

Note: no visibility symbol → assume default (package)

Methods

```
visibility  name(params)  :  ReturnType
```

```
+ getName() : String
+ setPrice(price : double) : void
+ Product(id : Long, name : String)
```

Constructors are written like methods with no return type

Static Members: Notation and Code

In UML

Underlined — belongs to the class itself, not to any one object.

- instanceCount : int

+ getInstanceCount() : int

Example: a shared counter tracking how many Products exist

```
public class Product {
    private static int instanceCount = 0;

    private Long id;

    public Product() {
        instanceCount++;
    }

    public static int getInstanceCount() {
        return instanceCount;
    }
}
```

`static` is a Java keyword, written before the type. It means: one shared copy per class, not one per object — every Product increments the same instanceCount.

LIVE CODE

LIVE

Calling a Static Member from a Test Class

```
public class ProductTest {

    public static void main(String[] args) {
        Product p1 = new Product();
        Product p2 = new Product();
        Product p3 = new Product();

        int count = Product.getInstanceCount();
        System.out.println("Instances created: " + count);
        // Instances created: 3
    }
}
```

NOTICE

`Product.getInstanceCount()`

is called on the class name, not on p1, p2 or p3.

All three objects share one instanceCount — creating p2 doesn't reset what p1 changed.

p1.getInstanceCount() would also compile, but calling a static method through an object is misleading style — always prefer the class name.

Full Notation Cheat Sheet

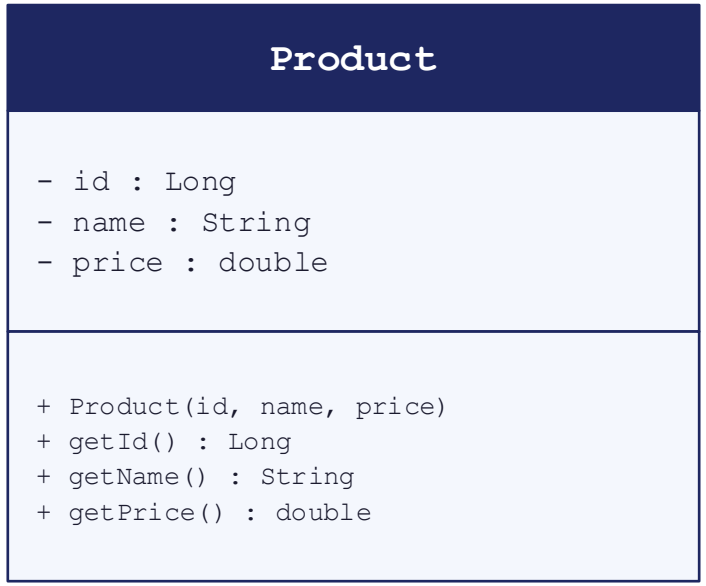
Visibility & Static — Quick Reference

```
+  public
-  private
#  protected
~  default
underline  =  static (Java keyword 'static', one shared copy per class)
```

LIVE ACTIVITY

LIVE

Live: Draw the Product UML Diagram



DO IT TOGETHER

- Draw the three-compartment box for Product
- Add the id, name, price attributes with correct visibility (-)
- Add the constructor and three getters with correct visibility (+)
- Now hand a partner a blank box — can they fill it in from your Product.java?

06

From Diagram to Code and Back

Design starts on paper, not the IDE

LO1 LO5

Translating in Both Directions

Diagram → Code

Given: - discountRate : double and + applyDiscount(rate : double) : void

Students write the matching Java field and method — correct visibility, correct types, correct signature.

Code → Diagram

Given: an existing Java class with private fields and public methods

Students draw the class box, correctly mapping each modifier to +, -, # or ~.

Design starts on paper — not in the IDE.

Professional developers sketch class relationships before writing a single line of code. UML is the shared vocabulary that lets a whole team agree on a design before anyone commits to an implementation.

This is exactly what your coursework report evidences under LO5.

QUICK CHECK

One More Round

Q1 **What does an underlined attribute in a UML class diagram mean?**

→ *It's a static member — shared by the class, not per-object.*

Q2 **How would you write 'private double price' in UML?**

→ *- price : double*

Q3 **Why draw the diagram before writing the code?**

→ *It surfaces design decisions and mistakes cheaply, before implementation cost is sunk.*

LECTURE RECAP

What We Covered Today

Access Modifiers

private, default, protected, public — and why fields should almost always be private.

The Spring Connection

Jackson needs getters to serialise JSON — we broke it, then fixed it, live.

JavaBeans & Immutability

getX()/setX() convention; setters only where mutation is genuinely needed.

Constructors

A no-arg constructor for frameworks, a full constructor for your own code.

UML Class Diagrams

Three compartments, visibility symbols, static underlining — the professional notation.

Diagram ⇄ Code

Fluent translation both directions — this is a coursework and LO5 skill.

UP NEXT

Your Tutorial 2 Tasks — in the next tutorial session

YOUR TASKS IN THE 2-HOUR LAB

1. Implement Product with private fields, full + no-arg constructors, and getters
2. Add a nested Address class embedded in a new Customer class
3. Draw the UML for both classes (any tool), showing the association
4. Verify: remove one getter, hit the endpoint, observe the empty JSON, restore it
5. Commit: 'Lecture 2: encapsulation and UML for Product and Customer'

RESOURCES

Baeldung

Guide to Java Encapsulation

Any diagram tool

draw.io, PlantUML, Mermaid, PowerPoint

Or by hand

a clear photo of a drawing is fine

Oracle Docs

Controlling Access to Members of a Class

Lecture 3 Preview: Collections, Generics & the Service Layer — building a real multi-product API.



Encapsulation, understood.

Build it in Tutorial 2, next week.